Exercice 2 IML

k-Nearest Neighbors (kNN) Analysis

1. Results Table

The table below shows the test accuracy scores for each combination of k and distance metric (L1 and L2):

```
Table of results :
k                    1         10        100       1000      3000
distance_metric
l1               0.967044  0.961718  0.923103  0.745007  0.401798
l2               0.968043  0.957723  0.920107  0.741678  0.398136
```

Figure 1: Test accuracy scores for $k \in \{1, 10, 100, 1000, 3000\}$ and distance metrics L1, L2.

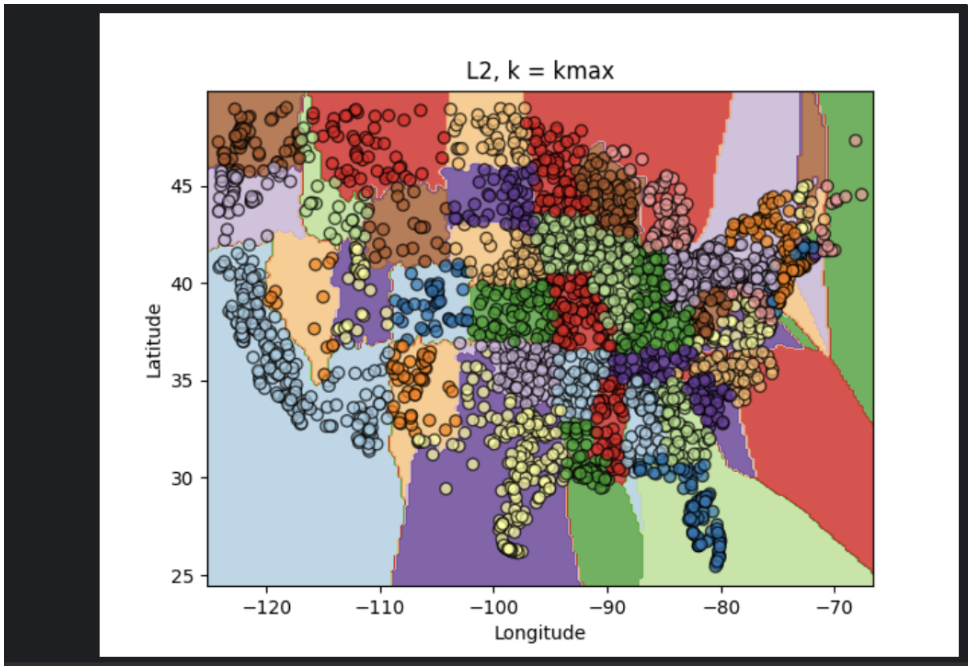
As k increases in k-nearest neighbors (kNN), the algorithm begins to include more neighbors in its decision-making process. While this can make the model more robust to noise for smaller k, it also increases the likelihood of including data points that belong to multiple classes. This can lead to higher misclassification rates, as the model prioritizes general trends over specific distinctions. Additionally, the results show minimal differences between the L1 (Manhattan) and L2 (Euclidean) distance metrics. This suggests that the choice of norm has a limited impact on accuracy in this context, likely because both distance metrics preserve the general neighborhood structure of the data.

2. Visualization and Analysis of Plots

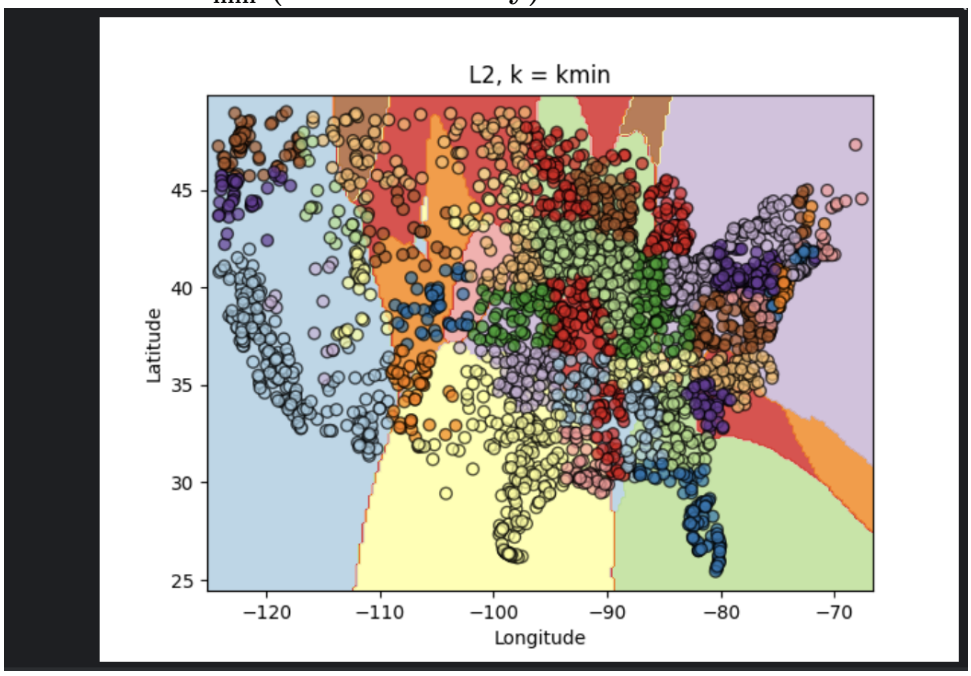
a. Included Plots

The following plots have been generated and scaled for readability:

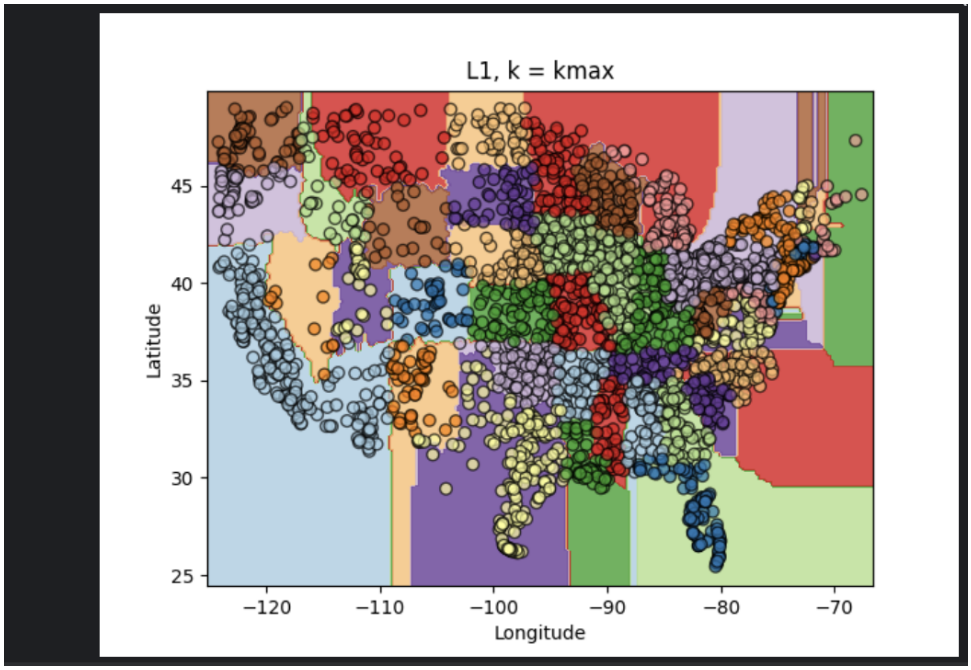
1. **L2 with $k = k_{\mathbf{max}}$ (highest accuracy)**



2. **L2 with $k = k_{\text{min}}$ (lowest accuracy)**



3. **L1 with $k = k_{\text{max}}$ (highest accuracy)**



b. Differences Between k_{max} and k_{min} with L2)

When comparing plots (i) and (ii):

- With $k = k_{\text{max}}$, the decision boundary is smoother, as the model averages contributions from a larger number of neighbors. This leads to better generalization and explains the higher accuracy. The decision boundary reflects a more global structure of the data rather than reacting to local fluctuations or noise. This allows the algorithm to generalize well, particularly in cases where each state has a substantial number of data points.

- With $k = k_{\text{min}}$, the decision boundary is more irregular and follows the local structure of the data more closely. This can lead to overfitting, as the model is more sensitive to outliers or noise in the training data, resulting in lower accuracy.

c. Effects of Distance Metrics on Classification (k_{max}, L1 vs L2)

When comparing plots (i) and (iii):

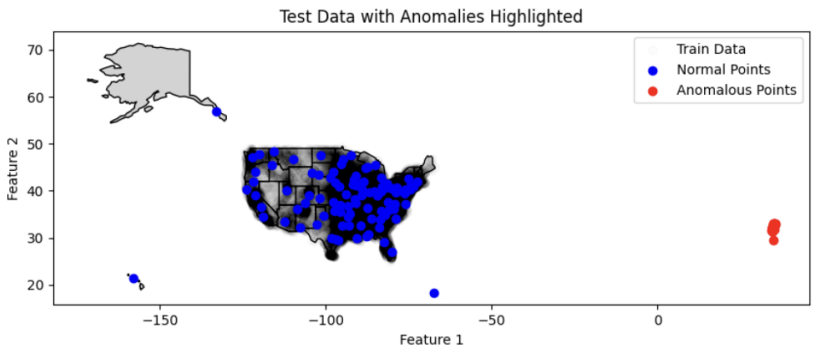
- The L2 (Euclidean) distance metric favors more circular decision boundaries because it calculates the straight-line distance between points, treating all directions equally. As a result, the boundaries are smoother and more gradual.

- The L1 (Manhattan) distance metric produces diamond-shaped boundaries, as it penalizes movement along each axis independently. This results in sharper, more angular decision boundaries (they divide the space into verticat and horizontal linee) compared to the smoother transitions seen with L2.

- These differences highlight how the choice of distance metric affects the classification space and decision boundaries. L2's smoothness often results in better accuracy when the data is well-distributed, whereas L1's angular boundaries may be beneficial when data is more structured along axes.

Anomaly Detection Using kNN

Generated Plot



Analysis of Anomalies

The anomalies detected by the model differ significantly from the normal data:

- Anomalies are typically located farther away from the dense clusters of normal data points.
- These points might correspond to outliers or rare patterns not represented in the training data.
- The model's reliance on distance metrics effectively isolates these points based on their dissimilarity to the training set.

Boosting

1. Graph of the accuracy

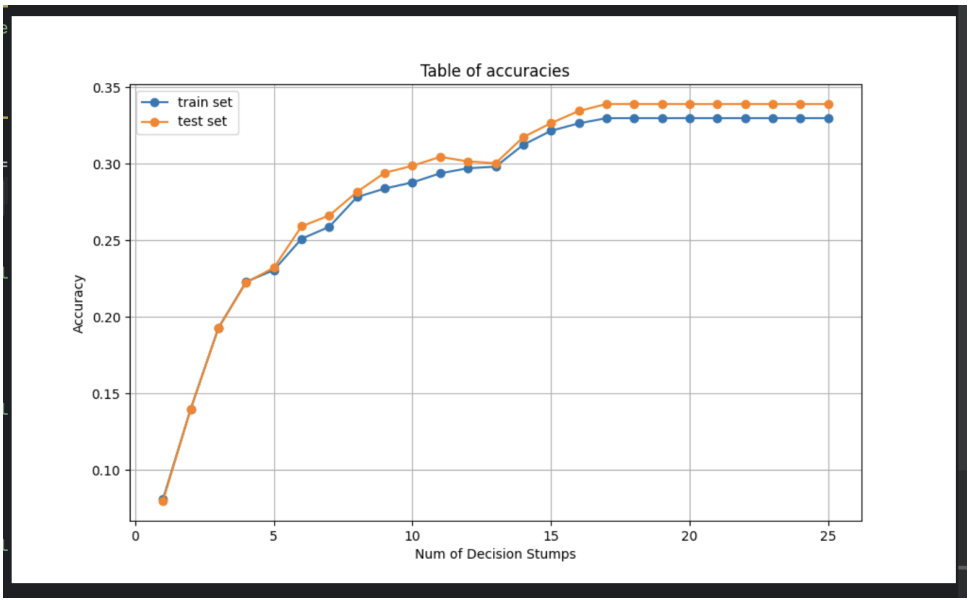


Figure 2: Accuracy vs. Number of Decision Stumps

2. Sample of predicted class for 1, 5, 15, 25.

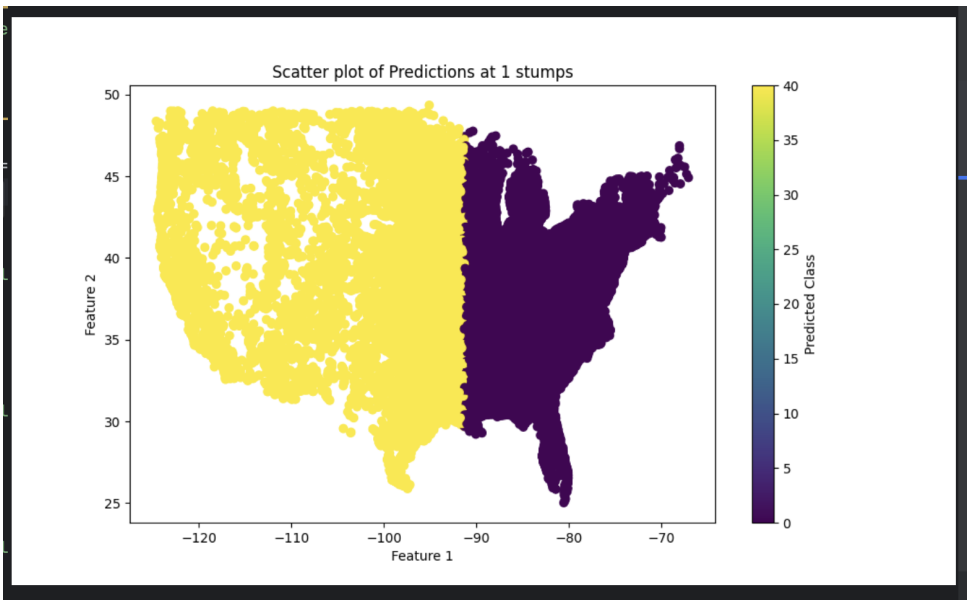


Figure 3: Training Data with Predicted Class for 1 Decision Stump

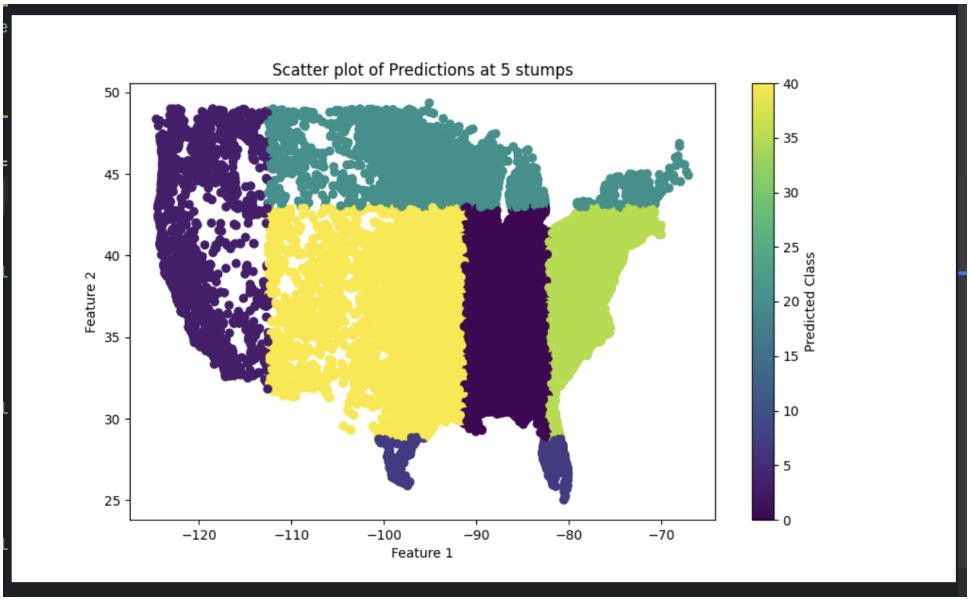


Figure 4: Training Data with Predicted Class for 5 Decision Stumps

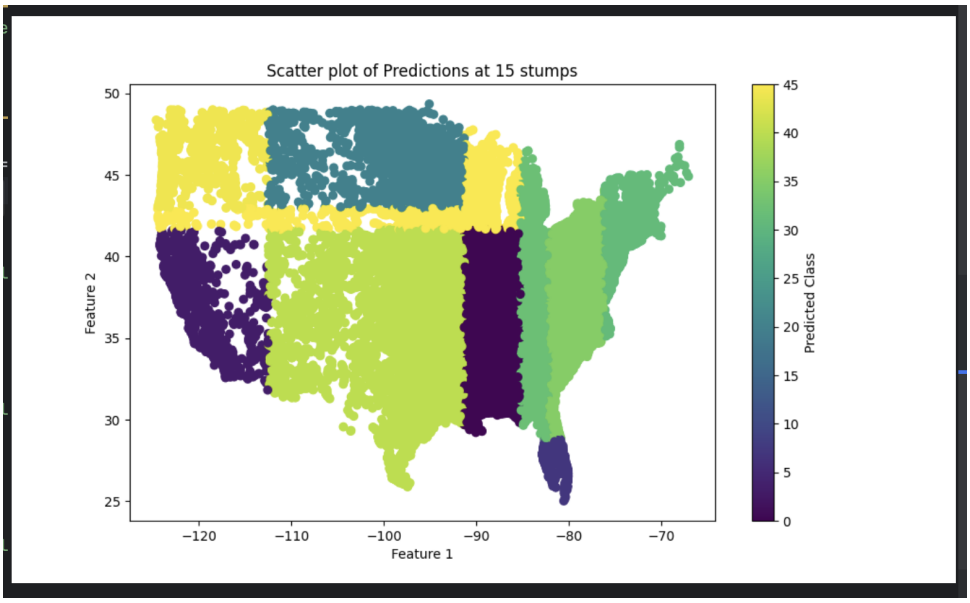


Figure 5: Training Data with Predicted Class for 15 Decision Stumps

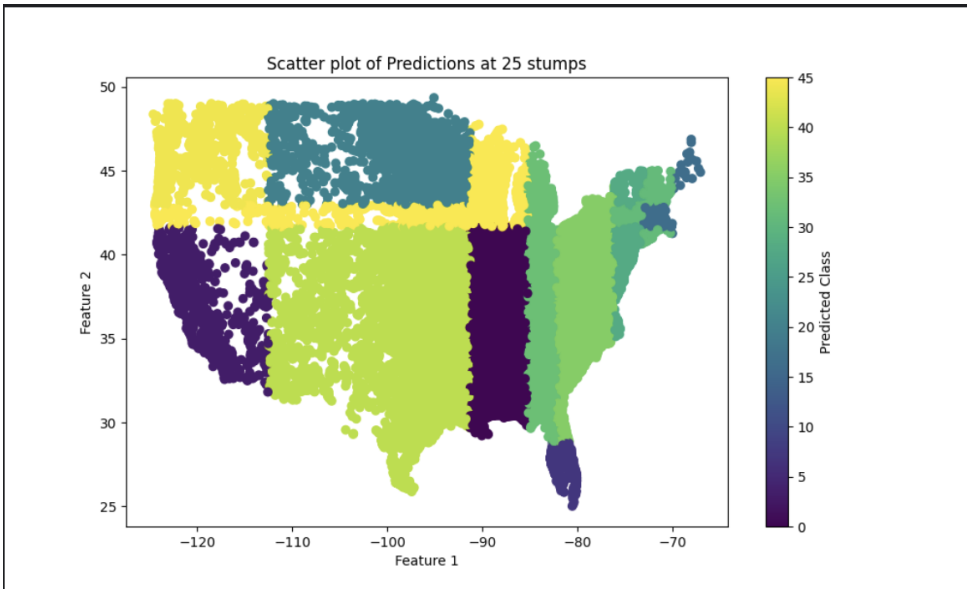


Figure 6: Training Data with Predicted Class for 25 Decision Stumps

Final accuracy

The final accuracy of the boosting method is likely not perfect due to the inherent limitations of decision stumps as weak learners. In our dataset, which consists of a large number of small and distinct states rather than a simple binary division, the simplicity of decision stumps poses a significant challenge. Each stump makes decisions based on a single feature, ignoring the complex interplay between features such as latitude and longitude, which are critical for accurate classification in this context.

Additionally, the boosting algorithm selects the feature that provides the best split at each stage, but this may not fully account for interactions between features or for the geographic and categorical subtleties of the data. The decision boundaries formed by combining stumps may not capture the granularity needed to distinguish between the many small states effectively. While boosting improves performance by iteratively focusing on harder-to-classify points, the reliance on weak learners limits the model's ability to achieve perfect accuracy.

Furthermore, the complexity of the dataset, including potential noise and feature dependencies, makes achieving 100% accuracy unrealistic. More advanced weak learners, such as decision trees with deeper splits, or other algorithms like SVMs or neural networks, might better capture the structure of the data. However, even with these models, challenges like noise, feature interactions, and the high number of distinct states would still make perfect classification difficult.

4. Compare kNN and Boosting

In the case of our dataset, which is based on U.S. states, the data is structured in a way that makes it highly suitable for tasks where a large number of points are available for each state. For instance, states with high population density, such as California, have ample data points, making it easier for algorithms like boosting to identify patterns effectively and classify data accurately.

When comparing boosting with decision stumps to k-Nearest Neighbors (KNN), the

performance difference largely depends on the nature of the data and the model's ability to generalize:

- **KNN** relies on memorizing the training data and predicts the label based on the majority class of the nearest neighbors. This can work well in scenarios with smaller or well-structured datasets. However, KNN might struggle with larger, noisier datasets or when dimensionality increases. For example, when distinguishing between states with sparse data points, KNN may not capture subtle patterns effectively.

- **Boosting** with decision stumps, on the other hand, is more efficient for tasks involving large datasets like ours. It iteratively improves predictions by focusing on misclassified examples and is less sensitive to noise compared to KNN. Although decision stumps are simple and only split data based on one feature, their combined effect in boosting can model the structured nature of our dataset effectively. For instance, it may use features like latitude to approximate geographical boundaries while leveraging other features, such as economic or demographic variables, to refine predictions further.

Boosting tends to outperform KNN in datasets like ours, where patterns are relatively well-defined and there are numerous data points per state. While KNN might have an advantage in more complex or sparse datasets, its reliance on local neighbors and sensitivity to dimensionality limits its scalability and efficiency.